



Pre-Joining Learning Path for React JS

Welcome to Zignuts Technolab. Thank you for joining us and we are hoping for a long-term association and a continuous journey of learning and growth.

At Zignuts, we continuously strive to become better and stay ahead of the technology trends with respect to IT services. We are constantly upgrading our skills and readiness for new challenges and we have built a culture of learning and development of individuals and the organization as well.

In this program, you will learn from the very basics of Javascript, from beginner to advanced level. The objective of this program is to make you ready to take on live projects and join the team of any project or work on a project assigned to you.

Important Notes - Read it carefully:

- You should have your corporate email created. Please talk to HR if not created.
- You will need to create an account on bitbucket.org. Bitbucket is our primary platform to store the source code of the project and version control. You will need to create git repositories in your account to save your code of any assignments that you will do during the training period.
- Make sure to take notes about the topics that you learn and revise them weekly.
- There will be periodic surprise tests and interviews to check your knowledge.
- Practice is very important during the training period and therefore you are open to work in extended working hours to enhance your skills. You are also allowed to practice during the weekends. You will need to inform HR 1 day prior to the weekend or holiday if you are planning to work from the office.
- You should do a periodic review of your progress to decide whether you need any extra hours for more practice.

- All the documents, training materials, videos, guides, code snippets and any code that you develop during the training period and even in your projects are PRIVATE and CONFIDENTIAL to Zignuts and they are not to be shared with anyone outside your team or unauthorized persons. Stringent actions will be taken for disclosing any information.
- There will be certain assignments during this training program. You will need to perform those assignments, present a demo and submit the code to your assigned training coordinator and supervisor.
- The evaluation criteria will consider your assignment completion, coding skills, presentation skills, UX skills, attitude, behavior with co-workers, behavior with seniors, speed and quality of learning, accuracy, and usability.
- You are required to submit a daily progress report via email to your training coordinator and supervisor.

Major Learning Goals

In this training program, you will learn the following topics.

- Git and best practices
- Basics of HTML
- Basics of CSS with Responsive
 - Practical Assignment (HTML + CSS + SCSS)
- Javascript Basic & Advance Concepts
- React.js - The Frontend
 - Assignments
- Quick Help
- Code Management (Standard Pattern for Branch and commit message)

Session 1 - Git and best practices

Estimated Time: 4-5 hours

Concepts/Points need to learn

- Why use Git?
- Installing Git
- How does git work?
- Creating a repository
- Staging Files
- Making Commits
- Undoing Things
- Branches
- Merging Branches
- Introduction to GitHub
- Collaborating on GitHub
- Forking

Reference Link: https://www.youtube.com/playlist?list=PL4cUxeGkcC9goXbgTDQ0n_4TBzOO0ocPR

Session 2 - HTML Learning

Estimated Time: 2 - 2.5 days

<https://www.w3schools.com/html/> (Recommended)

- XHTML Learning: <https://www.javatpoint.com/xhtml-tutorial>

Session 3 -CSS + SCSS Learning

Estimated Time: 2 - 2.5 days

<https://www.w3schools.com/css/default.asp> (Recommended)

SCSS: https://www.youtube.com/watch?v=_a5j7KoflTs (Recommended)

- Responsive Learning: <https://web.dev/learn/design/>
- Bootstrap Learning: <https://getbootstrap.com/>

Assignment (HTML + CSS/SCSS)

Estimated Time: 1 day

<https://templatemo.com/screenshots-720/template-576-snapx-photography.jpg>

Open this URL in you will get the screenshot of the Landing page so you have to create the web design of this (with Responsive as well)

Session 4 - Javascript Concepts

Estimated Time: 5 - 6 days

Tutorial (Each Concept Wise)

- <https://www.w3schools.com/js/> (Recommended)
- <https://javascript.info/>

Fundamentals Concepts of JavaScript

- <https://www.dottedsqureel.com/fundamentals-javascript/>
- <https://www.geeksforgeeks.org/7-javascript-concepts-that-every-developer-must-know/>

Javascript Practice

- <https://learnjavascript.online/>
- <https://www.jschallenger.com/>

Logical Exercise (Estimation Time: 1 - 1.5 hours)

Task 1: Need to create JavaScript functions for a sum of numbers in string (Example like "foo8bar8cat2tc2")

Task 2: Need to create a Javascript function for the sum of string (Example like "1.5, 2.3, 3.1, 4, 5.5, 6, 7, 8, 9, 10.9")

Session 5 - React

Estimated Time: 10 - 11 days

Documentation: <https://reactjs.org/docs/hello-world.html> & <https://beta.reactjs.org/>

React Complete Tutorial:

- <https://www.youtube.com/watch?v=bMknfKXIFA8>
- <https://www.youtube.com/watch?v=QFaFicGhPoM&list=PLC3y8-rFHvwwg3vaYJgHGnModB54rxOk3>
- <https://www.youtube.com/watch?v=Ke90Tje7VS0>

React Redux:

- Documentation: <https://react-redux.js.org/>
- Tutorial:  React Redux Tutorials - 1 - Introduction

REACT Assignments (Estimated Time: 3 days)

Assignment 1

Estimated Time: 0.5 days

=====

Requirements:

Convert the given design to a responsive web page using React JS.

Design link - <https://www.figma.com/file/qb0C6GI24GvJqMrE3wcyrt/Mendleson?node-id=0%3A1>

Adjust the responsiveness based on your experience and user-centric thinking

Adhere to the pixel perfect development of the given design

Use of scss is preferred over traditional css

Feel free to add effects, animations etc to make things look nicer

Feel free to follow any css/scss framework such as bootstrap, tailwind or any other you may decide based on your experience

Deliver the source code via Github or Bitbucket repository

Evaluation Criteria:

Completion of the given task and pixel perfect matching with the design

Project and coding structure and standards, commenting, naming conventions and clean coding practices

Components structure, reusability, separation of concerns for components and scss/css

=====

Assignment 2

Estimated Time: 0.5 days

=====

Develop a front-end application that would let the users help list and browse games developed for different platforms conveniently.

User this Web API to fetch the details:

Web API: <https://s3-ap-southeast-1.amazonaws.com/he-public-data/gamesarena274f2bf.json>

Utilize the parameters

- Title, platform, score, genre, editors_choice

Task 1: Visually interactive listing all the games

Task 2: Implement feature to sort games based on platform

Task 3: Autocomplete search with game title

Parameters means

- Title: title of the game
- Platform: name of the platform from where game designed
- Score: game rating score
- Genre: genre of game
- editors_choice : whether game is editor choice or not

***Note:**

- Use proper Basic HTML and CSS
- Proper use of life cycle of React
- User Proper Naming conventions, Commenting and variable name in components
- Use bootstrap or any custom HTML material design for creating more user friendly view
- Follow component-based approach so if you are creating listing Or Auto suggestion search Or filter each have represent with component

=====

Assignment 3

Estimate Time: 1 day

=====

You have to create the closest possible replica of the
<https://react-advanced-assignment.psamd.vercel.app/>

API for Data Fetch:

Method: GET

URL: <https://jsonplaceholder.typicode.com/users>

For Avatar, you will get from:

[https://avatars.dicebear.com/v2/avataaars/{{username}}.svg?options\[mood\]\[\]=happy](https://avatars.dicebear.com/v2/avataaars/{{username}}.svg?options[mood][]=happy)

Response you will get

[

```

{
  id,    // The user's id
  username,
  name,
  email,
  phone,
  website,
  address: {
    street, // Address line 1
    suite, // Address line 2
    city,
    zipcode
  },
  company: {
    name, // The name of company where the user works
  }
}
]

```

Functionalities that you have to perform:

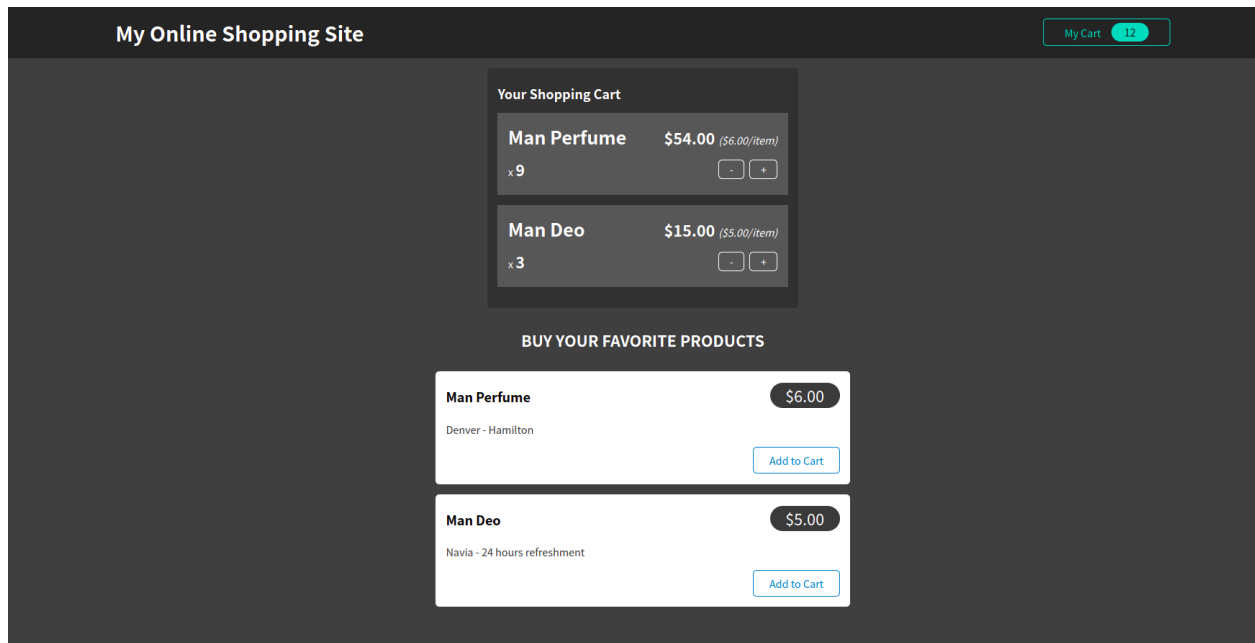
- Proper state management
- Like, Edit (with modal pop-up) , Remove specific frontend only
- One we edit the any data then it should reflect in list also (NOT IN API)

=====

Assignment 4 (Redux)

Estimated Time: 0.5 days

=====



Dummy Product JSON

```
const SAMPLE_PRODUCTS = [  
  {  
    id: 'p1',  
    price: 126,  
    title: 'Man Perfume',  
    description: 'Denver - Hamilton',  
  },  
  {  
    id: 'p2',  
    price: 125,  
    title: 'Man Deo',  
    description: 'Navia - 24 hours refreshment',  
  },  
];
```

Tasks requirement:

- Shopping card Web Application need to create with basic CSS & HTML
- Require to use Redux
- When I am selecting any product then it will store in Cart
- When I click on "My Cart" then details visible over product list (your view match with above screenshot)
- I can add/remove quantity from Shopping cart itself (also according to quantity price also modify)

- IF I am adding the same product again “Add to cart” and that product already exists in My cards then the quantity only needs to increase in “My card”

Quick Help

From tutorials/videos, you can able to understand and learn about the concept and flow but at the same time coding problems/suggestions, you can refer to some of the good websites like

- Stack Overflow
- Dev Community (A best and good way to discuss any concept of any language and you can also improve your writing skills as well.

Code Management & Branch Pattern

❖ Branch naming Pattern

As any project has divided into 3 phases testing, staging, and master so according we are creating branches like

For Live Server: master

For Staging Server: staging

For testing Server: development or simply as develop

When initial project most of the time we have started with UI development or theme integration so branch should not like "design" or "uidesign" but it should be like **<module_name>-<design/UI>** "authentication-UI", so from this, we can able to track individual module-specific UI pages.

Same format is applicable for API integration OR make the page as dynamic so the format should like **add-feature-<module_name>** (it include all operation coming in that module-specific only) like **add-feature-authentication**

For any bug fixes, you can create a specific branch like bug number/JIRA card no with format **<BUG_NUM/JIRACARD-fix-<bug_title>**(title should max 15 characters) So like **“786-fix-validation”** OR **“789-fix-homepage-CSS”**

❖ Commit Message

In Many projects, we have seen that Commit messages are provided like "build", "latest changes", "changes", "bugfix". It is not proper because If we have to track something then it is difficult, also not able to understand what you have pushed in this commit so now **YOU HAVE TO STRICTLY FOLLOW** certain patterns for commit messages like

For new feature: git commit -m "**feat(authentication): Add login, forgot password & reset password API integration**"

For update feature: git commit -m "**upda(authentication): Change Category Page API request params**"

For bug/fixes: git commit -m "**fix(homepage): multiple time API call performing whenever component load**"

❖ Branch naming convention for release

Whenever we release any feature or something in the master branch then the branch should be like **"landing-page-release-v1"** OR **"package-upgrade-release-v1"**, which means each release-wise branch will merge with master. Same thing if you are doing in staging server then **stag-landing-page-release-v1** OR **stag-package-upgrade-release-v1** same for testing server **dev-landing-page-release-v1** OR **dev-package-upgrade-release-v1**